

STAGE 0 - EDA & DATA INSPECTION

In [124...

```
# =====  
# STAGE 0 - SALVAGE COLUMN FIX  
# Recover trusted columns from shifted FIBA data  
# =====  
  
import pandas as pd  
import numpy as np  
  
pd.set_option("display.max_columns", None)  
pd.set_option("display.width", 140)  
  
# Load data  
df = pd.read_csv("merged_synergy_ncaa2.csv")  
  
df['league_adj_best'] = df['league_adj_best'].round(3)
```

In [125...

```
# =====  
# STAGE 0 - EDA & DATA INSPECTION  
# PURPOSE:  
# Initial exploration of the FIBA → NCAA player development dataset  
#  
# FILE ASSUMPTION:  
# output/final_fiba_ncaa.csv  
#  
# THIS BLOCK ANSWERS GUIDING QUESTIONS:  
# Q1: What does the overall player pool look like?  
# Q2: Are there missing values or structural data issues?  
# =====  
  
# Basic shape  
print("Dataset Shape:")  
print(df.shape)  
  
print("\nColumn Names:")  
print(df.columns.tolist())  
  
print("\nData Types:")  
print(df.dtypes)  
  
print("\nFirst 5 Rows:")  
display(df.head())  
  
print("\nMissing Values:")  
missing = df.isna().sum().sort_values(ascending=False)  
display(missing[missing > 0])  
  
print("\nDuplicate Rows:")  
print(df.duplicated().sum())  
  
print("# Players Who Made D1 vs # of Unique Players Who Made D1: ")  
print(df["made_d1"].sum(), '-----', df[df["made_d1"] == 1]["NAME"].nunique())
```

Dataset Shape:
(2211, 39)

Column Names:

['NAME', 'GP', 'ROLE', 'COUNTRY', 'FIBA_CLASS', 'FIBA_YEAR', 'POSS', 'PTS', 'PPP', 'AST', 'TO', 'FG%', '3FG_ATT', '3FG%', 'EFG%', 'TOT_REB', 'FT_ATT', 'FT%', 'STL_BLK', 'AST_TO', 'PTS/G_career', 'AST_career', 'TRB_career', 'STL_BLK_career', 'G_career', 'PER_career', 'best_season', 'bestteam', 'bestconf', 'PTS/G_best', 'AST_best', 'TRB_best', 'STL_BLK_best', 'PER_best', 'made_d1', 'bestteam_rank', 'team_mult', 'conf_mult', 'league_adj_best']

Data Types:

NAME	object
GP	int64
ROLE	object
COUNTRY	object
FIBA_CLASS	object
FIBA_YEAR	int64
POSS	float64
PTS	float64
PPP	object
AST	float64
TO	float64
FG%	object
3FG_ATT	float64
3FG%	object
EFG%	object
TOT_REB	float64
FT_ATT	float64
FT%	object
STL_BLK	float64
AST_TO	object
PTS/G_career	float64
AST_career	float64
TRB_career	float64
STL_BLK_career	float64
G_career	float64
PER_career	float64
best_season	object
bestteam	object
bestconf	object
PTS/G_best	float64
AST_best	float64
TRB_best	float64
STL_BLK_best	float64
PER_best	float64
made_d1	float64
bestteam_rank	float64
team_mult	float64
conf_mult	float64
league_adj_best	float64

dtype: object

First 5 Rows:

	NAME	GP	ROLE	COUNTRY	FIBA_CLASS	FIBA_YEAR	POSS	PTS	PPP	AST	TO	FG%
0	Yvonne Ejim	2	NaN	Canada	U19	2019	1.5	1.0	0.667	0.0	0.0	33.3%
1	Yvonne Ejim	7	Slashing wing	Canada	U19	2021	14.9	13.4	0.904	1.6	2.3	46.8%
2	Aaliyah Edwards	7	Slashing wing	Canada	U19	2019	13.1	11.1	0.848	1.7	2.1	47.5%
3	Avery Howell	7	Dynamic shooting wing	Canada	U19	2025	11.0	14.3	1.299	1.7	0.9	53.3%
4	Toby Fournier	7	Rim-finishing big	Canada	U17	2022	15.0	13.3	0.886	1.3	0.7	45.6%

Missing Values:

```
league_adj_best      1698
PTS/G_career         1698
TRB_career           1698
STL_BLK_career       1698
G_career             1698
best_season          1698
bestteam             1698
bestconf             1698
PTS/G_best           1698
AST_best             1698
TRB_best             1698
STL_BLK_best         1698
PER_best             1698
AST_career           1698
PER_career           1657
ROLE                 605
bestteam_rank        56
team_mult            56
conf_mult            56
made_d1              50
```

dtype: int64

Duplicate Rows:

0

Players Who Made D1 vs # of Unique Players Who Made D1:

561.0 ----- 321

Data Cleaning - Dtypes

```
In [126... import numpy as np

# Columns that should become numeric
numeric_cols = [
    "PPP",
    "FG%",
    "3FG%",
    "EFG%",
    "FT%",
    "AST_T0"
]

# Clean and convert
for col in numeric_cols:
```

```

# convert to string first
df[col] = df[col].astype(str)

# remove percent signs
df[col] = df[col].str.replace("%", "", regex=False)

# replace dash placeholders with NaN
df[col] = df[col].replace("-", np.nan)

# convert to numeric
df[col] = pd.to_numeric(df[col], errors="coerce")

# OPTIONAL:
# if you want percentages as decimals instead of whole numbers
percent_cols = ["FG%", "3FG%", "EFG%", "FT%"]

df[percent_cols] = (df[percent_cols] / 100).round(3)

# check result
print(df.dtypes)

```

NAME	object
GP	int64
ROLE	object
COUNTRY	object
FIBA_CLASS	object
FIBA_YEAR	int64
POSS	float64
PTS	float64
PPP	float64
AST	float64
T0	float64
FG%	float64
3FG_ATT	float64
3FG%	float64
EFG%	float64
TOT_REB	float64
FT_ATT	float64
FT%	float64
STL_BLK	float64
AST_T0	float64
PTS/G_career	float64
AST_career	float64
TRB_career	float64
STL_BLK_career	float64
G_career	float64
PER_career	float64
best_season	object
bestteam	object
bestconf	object
PTS/G_best	float64
AST_best	float64
TRB_best	float64
STL_BLK_best	float64
PER_best	float64
made_d1	float64
bestteam_rank	float64
team_mult	float64
conf_mult	float64
league_adj_best	float64
dtype:	object

```
# =====
# STAGE 0 – DATA CLEANING FIX FOR CLUSTERING
# NUMERIC CONVERSION OF PERCENT / TEXT COLUMNS
#
# ISSUE:
# Columns like FG%, 3FG%, EFG%, FT% contain strings:
# "45.6%", "-", etc.
#
# THIS BLOCK REPLACES THE PREVIOUS FEATURE PREP SECTION

# -----
# FEATURES USED FOR CLUSTERING
# -----
cluster_features = [
    'GP', 'POSS', 'PTS', 'PPP', 'AST', 'TO', 'FG%', '3FG_ATT', '3FG%', 'EFG%',
    'TOT_REB', 'FT_ATT', 'FT%', 'STL_BLK', 'AST_TO',
]

X = df[cluster_features].copy()

print("Cleaned Feature Data:")
display(X.head())

print("\nRemaining Missing Values:")
print(X.isna().sum())
```

Cleaned Feature Data:

	GP	POSS	PTS	PPP	AST	TO	FG%	3FG_ATT	3FG%	EFG%	TOT_REB	FT_ATT	FT%	STL_
0	2	1.5	1.0	0.667	0.0	0.0	0.333	0.0	NaN	0.333	3.0	0.0	NaN	
1	7	14.9	13.4	0.904	1.6	2.3	0.468	0.6	0.000	0.468	9.3	3.6	0.880	
2	7	13.1	11.1	0.848	1.7	2.1	0.475	0.4	0.333	0.484	6.6	4.9	0.559	
3	7	11.0	14.3	1.299	1.7	0.9	0.533	5.3	0.486	0.683	3.4	3.3	0.783	
4	7	15.0	13.3	0.886	1.3	0.7	0.456	0.0	NaN	0.456	11.3	6.6	0.457	

Remaining Missing Values:

```
GP          0
POSS        0
PTS         0
PPP         9
AST         0
TO          0
FG%        23
3FG_ATT     0
3FG%       304
EFG%        23
TOT_REB     0
FT_ATT      0
FT%       286
STL_BLK     0
AST_TO     50
dtype: int64
```

Handle Missing Values

In [128...

```
# =====  
# HANDLE MISSING VALUES  
# =====  
  
# -----  
# SHOOTING PERCENTAGES  
# Missing usually means no attempts  
# -----  
shooting_cols = ["3FG%", "FT%", "FG%", "EFG%"]  
  
for col in shooting_cols:  
    X[col] = X[col].fillna(0)  
  
# -----  
# PPP  
# small amount missing  
# -----  
X["PPP"] = X["EFG%"] * 2  
X["PPP"] = X["PPP"].fillna(X["PPP"].median())  
  
# -----  
# AST_TO  
# undefined often because turnovers = 0  
# use median or capped value  
# -----  
X["AST_TO"] = X["AST_TO"].fillna(X["AST_TO"].median())  
  
# -----  
# FINAL CHECK  
# -----  
print(X.isna().sum())
```

```
GP          0  
POSS        0  
PTS         0  
PPP         0  
AST         0  
TO          0  
FG%         0  
3FG_ATT     0  
3FG%        0  
EFG%        0  
TOT_REB     0  
FT_ATT      0  
FT%         0  
STL_BLK     0  
AST_TO      0  
dtype: int64
```

STAGE 1 - Segmentation

In [129...

```
# =====  
# STAGE 1 – UNSUPERVISED LEARNING ON ENTIRE FIBA DATASET  
# PLAYER ARCHETYPE DISCOVERY  
#  
# PURPOSE:  
# Identify natural player clusters/archetypes from FIBA statistical profiles
```

```

#
# THIS BLOCK ANSWERS GUIDING QUESTIONS:
# Q1: What distinct player archetypes exist in the full dataset?
# Q2: How many clusters best represent the player pool?
# Q3: Which features define each cluster?
#
# METHODS:
# - Standardization
# - KMeans clustering
# - Elbow Method
# - Silhouette Score
#
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score
import matplotlib.pyplot as plt

# Standardize
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# print nan values
print("NaN values after cleaning:")
print(np.isnan(X_scaled).sum())
# print cols with nan values
nan_cols = X.columns[X.isna().any()].tolist()
print("Columns with NaN values:", nan_cols)

```

NaN values after cleaning:
0
Columns with NaN values: []

In [130]...

```

# -----
# TEST MULTIPLE K VALUES
# -----
inertia = []
sil_scores = []

k_values = range(2, 9)

for k in k_values:
    model = KMeans(n_clusters=k, random_state=42, n_init=20)
    labels = model.fit_predict(X_scaled)

    inertia.append(model.inertia_)
    sil_scores.append(silhouette_score(X_scaled, labels))

# -----
# VISUALIZE ELBOW + SILHOUETTE
# -----
fig, ax = plt.subplots(1, 2, figsize=(8,4))

ax[0].plot(k_values, inertia, marker="o")
ax[0].set_title("Elbow Method")
ax[0].set_xlabel("Number of Clusters")
ax[0].set_ylabel("Inertia")

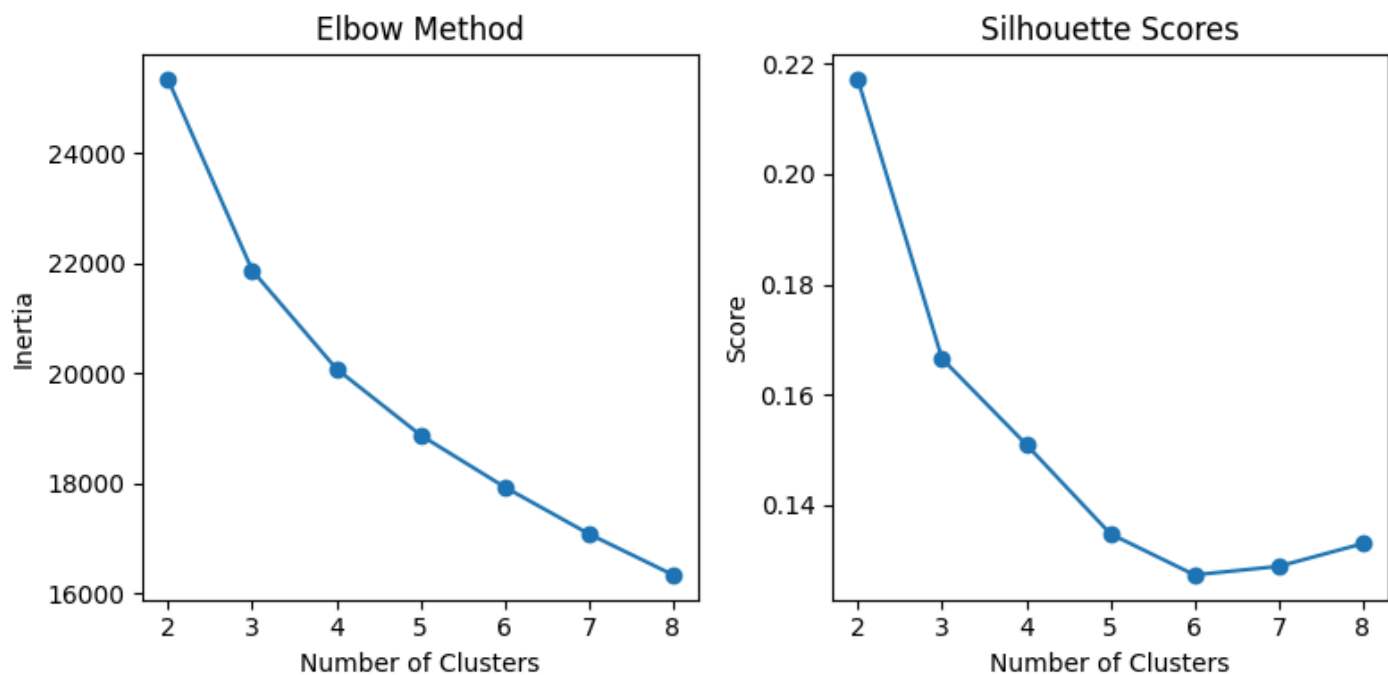
ax[1].plot(k_values, sil_scores, marker="o")
ax[1].set_title("Silhouette Scores")
ax[1].set_xlabel("Number of Clusters")
ax[1].set_ylabel("Score")

```

```
plt.tight_layout()
plt.show()

# -----
# PRINT RESULTS TABLE
# -----
results = pd.DataFrame({
    "k": list(k_values),
    "inertia": inertia,
    "silhouette_score": sil_scores
})

display(results)
```



	k	inertia	silhouette_score
0	2	25331.002926	0.217118
1	3	21864.392785	0.166511
2	4	20077.058376	0.150966
3	5	18871.238588	0.134757
4	6	17927.545334	0.127314
5	7	17077.124695	0.128860
6	8	16332.540668	0.132990

Final Cluster Model

```
In [131]: # =====
# STAGE 1 – FINAL CLUSTER MODEL (K = 3)
# PLAYER ARCHETYPE DISCOVERY ACROSS FULL FIBA DATASET
#
# THIS BLOCK ANSWERS GUIDING QUESTIONS:
# Q1: What distinct player archetypes exist in the full dataset?
# Q2: How many clusters best represent the player pool?
# Q3: Which features define each cluster?
```



```

#
# NOTE:
# Silhouette suggested K=2, but we intentionally use K=3 for richer
# basketball interpretation and more useful archetype separation.
# =====

# -----
# FIT FINAL KMEANS MODEL
# -----
kmeans = KMeans(
    n_clusters=3,
    random_state=42,
    n_init=25
)

df["fiba_cluster"] = kmeans.fit_predict(X_scaled)

# -----
# CLUSTER SIZE DISTRIBUTION
# -----
print("Cluster Counts:")
display(df["fiba_cluster"].value_counts().sort_index())

print("Cluster Share (%):")
display(
    (df["fiba_cluster"]
     .value_counts(normalize=True)
     .sort_index()
     * 100).round(1)
)

# -----
# CLUSTER CENTROIDS (STANDARDIZED SPACE)
# tells which features are above/below average
# -----
centroids_scaled = pd.DataFrame(
    kmeans.cluster_centers_,
    columns=cluster_features
)

print("Cluster Centroids (Z-Scores):")
display(centroids_scaled.round(2))

```

Cluster Counts:

fiba_cluster

0 647

1 572

2 992

Name: count, dtype: int64

Cluster Share (%):

fiba_cluster

0 29.3

1 25.9

2 44.9

Name: proportion, dtype: float64

Cluster Centroids (Z-Scores):

	GP	POSS	PTS	PPP	AST	TO	FG%	3FG_ATT	3FG%	EFG%	TOT_REB	FT_ATT	FT%
0	0.27	1.21	1.14	0.23	1.00	0.89	0.21	0.77	0.33	0.23	0.81	0.94	0.41
1	-0.50	-0.73	-0.92	-1.16	-0.58	-0.46	-1.10	-0.47	-0.68	-1.16	-0.59	-0.53	-0.49
2	0.11	-0.37	-0.21	0.52	-0.32	-0.32	0.50	-0.23	0.18	0.52	-0.19	-0.30	0.01

Cluster Profiles

In [132...

```
# -----
# CLUSTER PROFILE IN ORIGINAL SCALE
# easier basketball interpretation
# -----

cluster_profile = (
    df.groupby("fiba_cluster")[cluster_features]
      .mean()
      .round(2)
)

print("Cluster Means (Original Scale):")
display(cluster_profile)

# -----
# FEATURE DIFFERENCES ACROSS CLUSTERS
# which stats vary most by cluster
# -----

feature_spread = (
    cluster_profile.max(axis=0) - cluster_profile.min(axis=0)
).sort_values(ascending=False)

print("Most Differentiating Features:")
display(feature_spread.round(2))

# -----
# OPTIONAL: SAMPLE PLAYERS FROM EACH CLUSTER
# helps label archetypes
# -----
for c in sorted(df["fiba_cluster"].unique()):
    print(f"\nCLUSTER {c} SAMPLE PLAYERS")
    display(
        df[df["fiba_cluster"] == c][
            ["NAME", "COUNTRY", "FIBA_YEAR", "ROLE", "PTS", "PPP", "EFG%", "POSS"]
        ].sample(10)
    )
```

Cluster Means (Original Scale):

	GP	POSS	PTS	PPP	AST	TO	FG%	3FG_ATT	3FG%	EFG%	TOT_REB	FT_ATT	
fiba_cluster													
0	6.58	13.73	10.29	0.74	2.38	2.59	0.37	3.29	0.26	0.42	5.16	3.04	
1	5.07	4.68	1.89	0.38	0.61	1.18	0.20	1.08	0.09	0.21	1.78	0.82	
2	6.28	6.39	4.79	0.76	0.90	1.32	0.42	1.51	0.27	0.46	2.76	1.16	

Most Differentiating Features:

POSS 9.05
PTS 8.40
TOT_REB 3.38
FT_ATT 2.22
3FG_ATT 2.21
AST 1.77
STL_BLK 1.51
GP 1.51
TO 1.41
AST_TO 0.42
PPP 0.38
EFG% 0.25
FG% 0.22
3FG% 0.18
FT% 0.11
dtype: float64
CLUSTER 0 SAMPLE PLAYERS

	NAME	COUNTRY	FIBA_YEAR	ROLE	PTS	PPP	EFG%	POSS
21	Stephanie Visscher	Sweden	2019	Slashing wing	8.7	0.859	0.500	10.1
88	Eleonora Villa	Italy	2024	Dynamic shooting wing	13.4	0.883	0.467	15.2
790	Isaline Alexander	Canada	2019	Playmaking big	7.6	0.603	0.342	12.6
830	Zara Valcic	Croatia	2023	Playmaking wing	8.9	0.984	0.615	9.0
1928	Marina Marcetic	Serbia	2023	Secondary ball handler	5.9	0.577	0.405	10.1
573	Isobel Borlase	Australia	2022	Secondary ball handler	13.2	1.082	0.612	12.2
258	Lemyah Hylton	Canada	2022	Playmaking wing	8.0	0.533	0.269	15.0
951	Katerina Ziethammerova	Czechia	2019	Playmaking ball handler	5.7	0.453	0.270	12.5
425	Lucija Banovic	Croatia	2022	Slashing wing	11.1	0.780	0.473	14.3
276	Valentyna Kadlecova	Czechia	2022	Dynamic shooting wing	11.8	0.779	0.417	15.1

CLUSTER 1 SAMPLE PLAYERS

	NAME	COUNTRY	FIBA_YEAR	ROLE	PTS	PPP	EFG%	POSS	
648	Micaelly Yasmin De Freitas Luiz	Brazil	2023		NaN	0.3	0.125	0.200	2.7
1024	Marie Zilova	Czechia	2022		NaN	0.0	0.000	0.000	8.0
332	Lucia Decortes	Italy	2019		NaN	0.7	0.217	0.059	3.3
808	Avery Arije	Canada	2024		NaN	1.6	0.400	0.214	4.0
1790	Carly Ohia	New Zealand	2023		NaN	1.8	0.529	0.250	3.4
412	Peppi Lohko	Finland	2021	Stretch big	3.0	0.441	0.292		6.8
1095	Nicole Pihamaa	Finland	2023		NaN	2.3	0.778	0.333	3.0
1047	Karolina Petlanova	Czechia	2025	Dynamic shooting wing	3.4	0.444	0.267		7.7
388	Samantha Mullock	Great Britain	2025		NaN	0.7	0.333	0.250	2.1
701	Desislava Kantardzhieva	Bulgaria	2019		NaN	1.2	0.400	0.273	3.0

CLUSTER 2 SAMPLE PLAYERS

	NAME	COUNTRY	FIBA_YEAR		ROLE	PTS	PPP	EFG%	POSS
1369	Maja Manten	Germany	2025		Rim-finishing big	2.9	0.741	0.531	3.9
576	Ella Gordon	Australia	2022		NaN	3.0	1.125	0.667	2.7
950	Adela Burtova	Czechia	2019		Dynamic shooting wing	6.7	0.691	0.391	9.7
1255	Victoria Poros	Germany	2019		Spot-up shooting wing	3.9	0.796	0.574	4.9
1148	Jessi Nenonen	Finland	2025		Slashing wing	6.6	0.719	0.373	9.1
1015	Eliska Brejchova	Czechia	2022		Dynamic shooting wing	8.1	0.890	0.455	9.1
1365	Maria Perner	Germany	2025		Dynamic shooting wing	6.6	0.920	0.467	7.1
1992	Marina Asensio	Spain	2022		Playmaking wing	3.9	0.563	0.329	6.9
1956	Paula Bellot	Spain	2021		NaN	4.4	0.917	0.457	4.8
1602	Artemis Kouki	Greece	2023		Slashing wing	4.4	0.880	0.417	5.0

Pre Model Summary Section

In [133...

```
# =====  
# DATASET OVERVIEW  
# =====  
  
print("TOTAL ROWS:")  
print(len(df))  
  
print("\nUNIQUE PLAYERS:")  
print(df["NAME"].nunique())  
  
print("\nROWS WITH made_d1 = 1:")  
print(df["made_d1"].sum())
```

```

print("\nCLASS DISTRIBUTION:")
display(df["made_d1"].value_counts())

print("\nCLASS PERCENTAGES:")
display(
    (df["made_d1"].value_counts(normalize=True) * 100).round(2)
)

```

TOTAL ROWS:

2211

UNIQUE PLAYERS:

1381

ROWS WITH made_d1 = 1:

561.0

CLASS DISTRIBUTION:

made_d1

0.0 1600

1.0 561

Name: count, dtype: int64

CLASS PERCENTAGES:

made_d1

0.0 74.04

1.0 25.96

Name: proportion, dtype: float64

```

In [134... player_row_counts = df["NAME"].value_counts()

print("Average rows per player:")
print(round(player_row_counts.mean(), 2))

print("\nPlayers with multiple entries:")
print((player_row_counts > 1).sum())

print("\nMax entries for one player:")
print(player_row_counts.max())

```

Average rows per player:

1.6

Players with multiple entries:

541

Max entries for one player:

6

```

In [135... # unique players only
player_level = (
    df.groupby("NAME")["made_d1"]
      .max()
      .reset_index()
)

unique_players = len(player_level)

unique_d1_players = player_level["made_d1"].sum()

print("UNIQUE PLAYERS:")
print(unique_players)

```

```
print("\nUNIQUE D1 PLAYERS:")
print(unique_d1_players)

print("\nPERCENT OF PLAYERS WHO MADE D1:")
print(round((unique_d1_players / unique_players) * 100, 2), "%")
```

UNIQUE PLAYERS:
1381

UNIQUE D1 PLAYERS:
321.0

PERCENT OF PLAYERS WHO MADE D1:
23.24 %

In [136...

```
print("ROWS BY FIBA CLASS:")
display(df["FIBA_CLASS"].value_counts())

print("\nUNIQUE PLAYERS BY FIBA CLASS:")
display(
    df.groupby("FIBA_CLASS")["NAME"]
      .nunique()
      .sort_values(ascending=False)
)
```

ROWS BY FIBA CLASS:
FIBA_CLASS
U18 806
U16 642
U20 578
U19 103
U17 82
Name: count, dtype: int64
UNIQUE PLAYERS BY FIBA CLASS:
FIBA_CLASS
U18 679
U16 547
U20 446
U19 96
U17 79
Name: NAME, dtype: int64

Country-Level EDA Analysis

In [137...

```
# -----
# COUNTRY D1 CONVERSION RATE
# -----

country_conversion = (
    df.groupby("COUNTRY")["made_d1"]
      .mean()
      .sort_values(ascending=False)
)

top_conversion = (country_conversion * 100).round(2).head(20)
# add a mean conversion rate line to the plot
mean_conversion = (df["made_d1"].mean() * 100).round(2)

plt.figure(figsize=(10,7))

plt.barh(
```

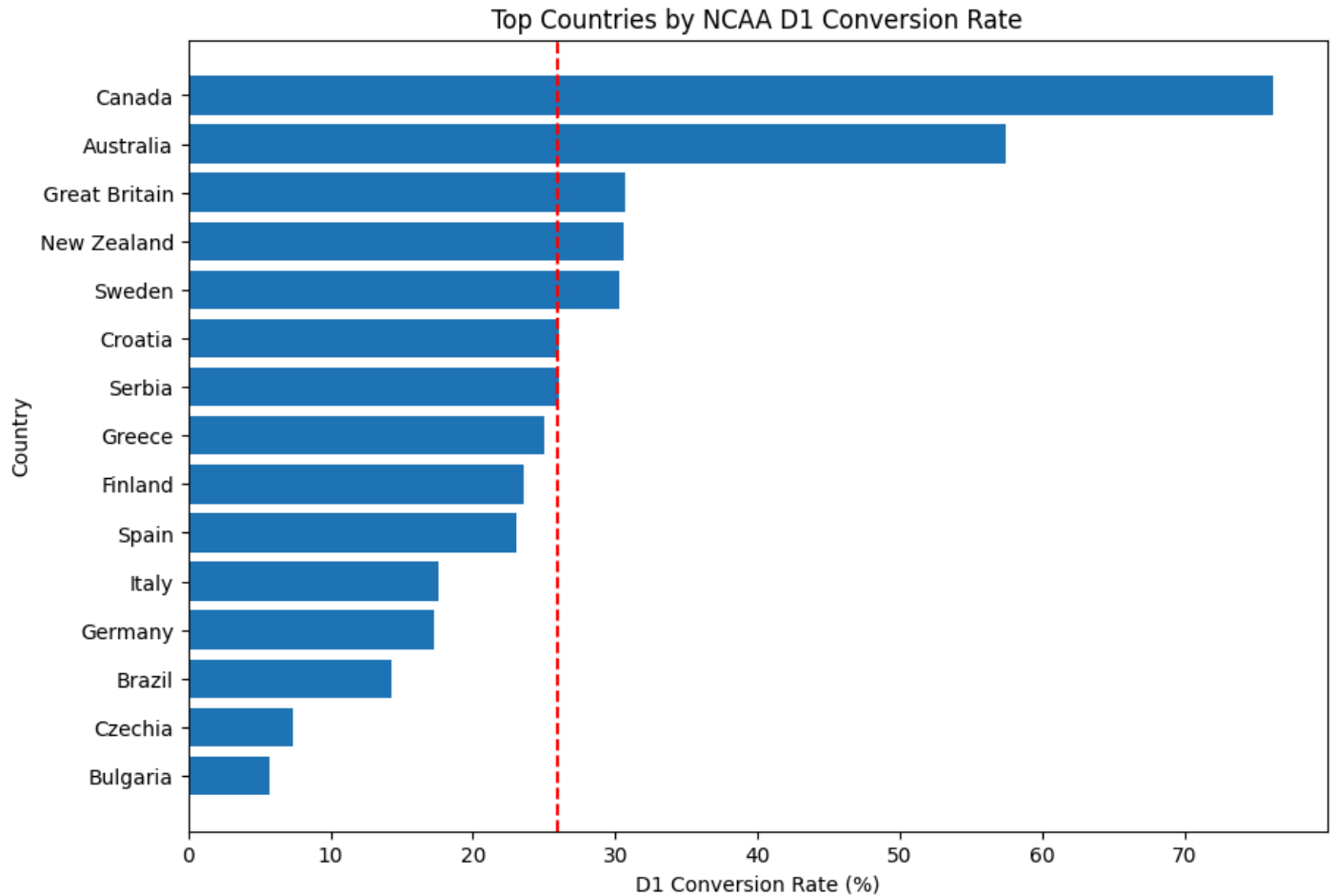
```

    top_conversion.index[::-1],
    top_conversion.values[::-1]
)
plt.axvline(mean_conversion, color="red", linestyle="--", label=f"Mean: {mean_conversion}")

plt.xlabel("D1 Conversion Rate (%)")
plt.ylabel("Country")
plt.title("Top Countries by NCAA D1 Conversion Rate")

plt.show()

```



```

In [138... # print how many players from each country in dataset
print("Player Counts by Country:")
display(
    df["COUNTRY"].value_counts().sort_values(ascending=False)
)

```

Player Counts by Country:

COUNTRY	
Germany	246
Greece	204
Italy	192
Czechia	177
Serbia	167
Finland	161
Sweden	155
Great Britain	153
Bulgaria	141
Spain	137
Canada	132
Australia	108
Croatia	92
New Zealand	73
Brazil	73

Name: count, dtype: int64

In [139...

```
# -----
# COUNTRY NCAA IMPACT
# -----

country_impact = (
    df.groupby("COUNTRY")["league_adj_best"]
      .mean()
      .sort_values(ascending=False)
)

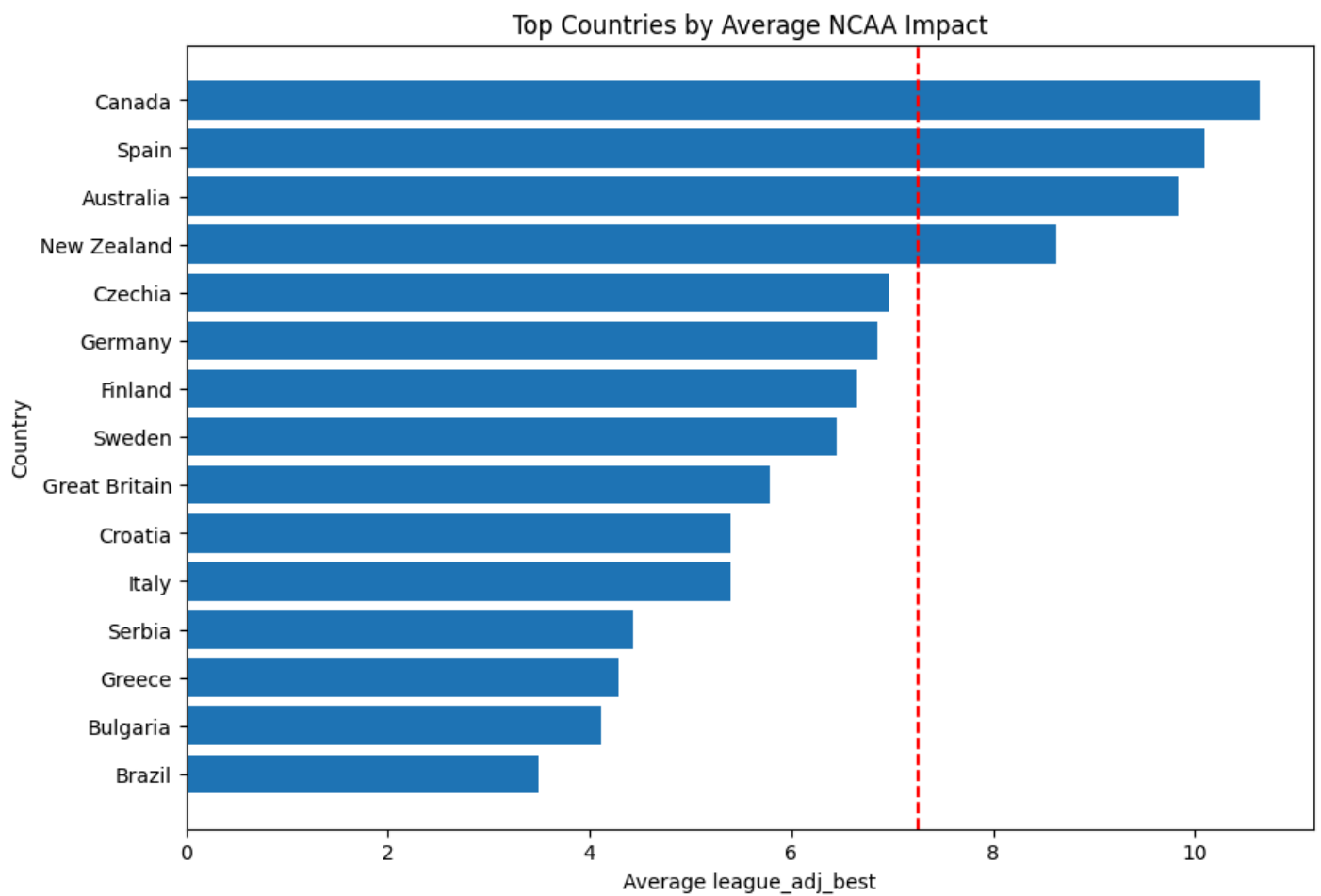
top_impact = country_impact.head(20)

plt.figure(figsize=(10,7))

plt.barh(
    top_impact.index[::-1],
    top_impact.values[::-1]
)

mean_impact = df["league_adj_best"].mean().round(2)
plt.axvline(mean_impact,color = 'red', linestyle = '--', label = f"Mean Impact: {mean_im
plt.xlabel("Average league_adj_best")
plt.ylabel("Country")
plt.title("Top Countries by Average NCAA Impact")

plt.show()
```

```
In [140... country_summary = (
    df.groupby("COUNTRY")
      .agg(
        total_players=("made_d1", "count"),
        d1_rate=("made_d1", "mean")
      )
)

country_summary = country_summary[
    country_summary["total_players"] >= 15
]

country_summary = country_summary.sort_values(
    "d1_rate",
    ascending=False
)

plt.figure(figsize=(8,6))

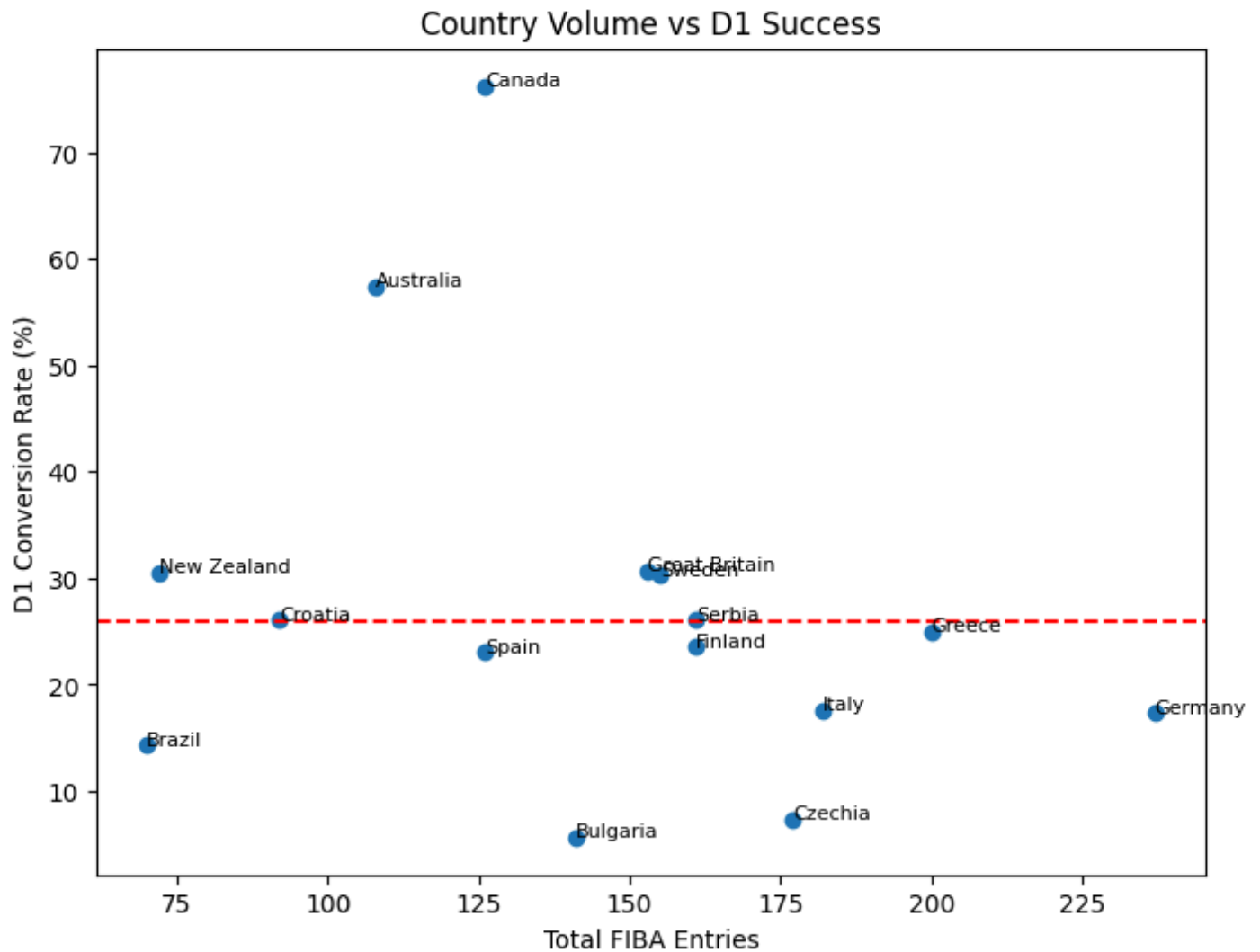
plt.scatter(
    country_summary["total_players"],
    country_summary["d1_rate"] * 100
)

for country in country_summary.index:
    plt.text(
        country_summary.loc[country, "total_players"],
        country_summary.loc[country, "d1_rate"] * 100,
        country,
        fontsize=8
    )
```

```
plt.axhline(mean_conversion, color="red", linestyle="--", label=f"Mean D1 Conversion: {m

plt.xlabel("Total FIBA Entries")
plt.ylabel("D1 Conversion Rate (%)")
plt.title("Country Volume vs D1 Success")

plt.show()
```



Average FIBA Stats for D1 Players

```
In [141... compare_stats = [
    "PTS",
    "PPP",
    "AST",
    "TOT_REB",
    "3FG_ATT",
    "FG%",
    "FT%",
    "STL_BLK",
    "AST_TO"
]

d1_summary = (
    df.groupby("made_d1")[compare_stats]
    .mean()
)

d1_summary.T.plot()
```

```

kind="bar",
figsize=(10,6)
)

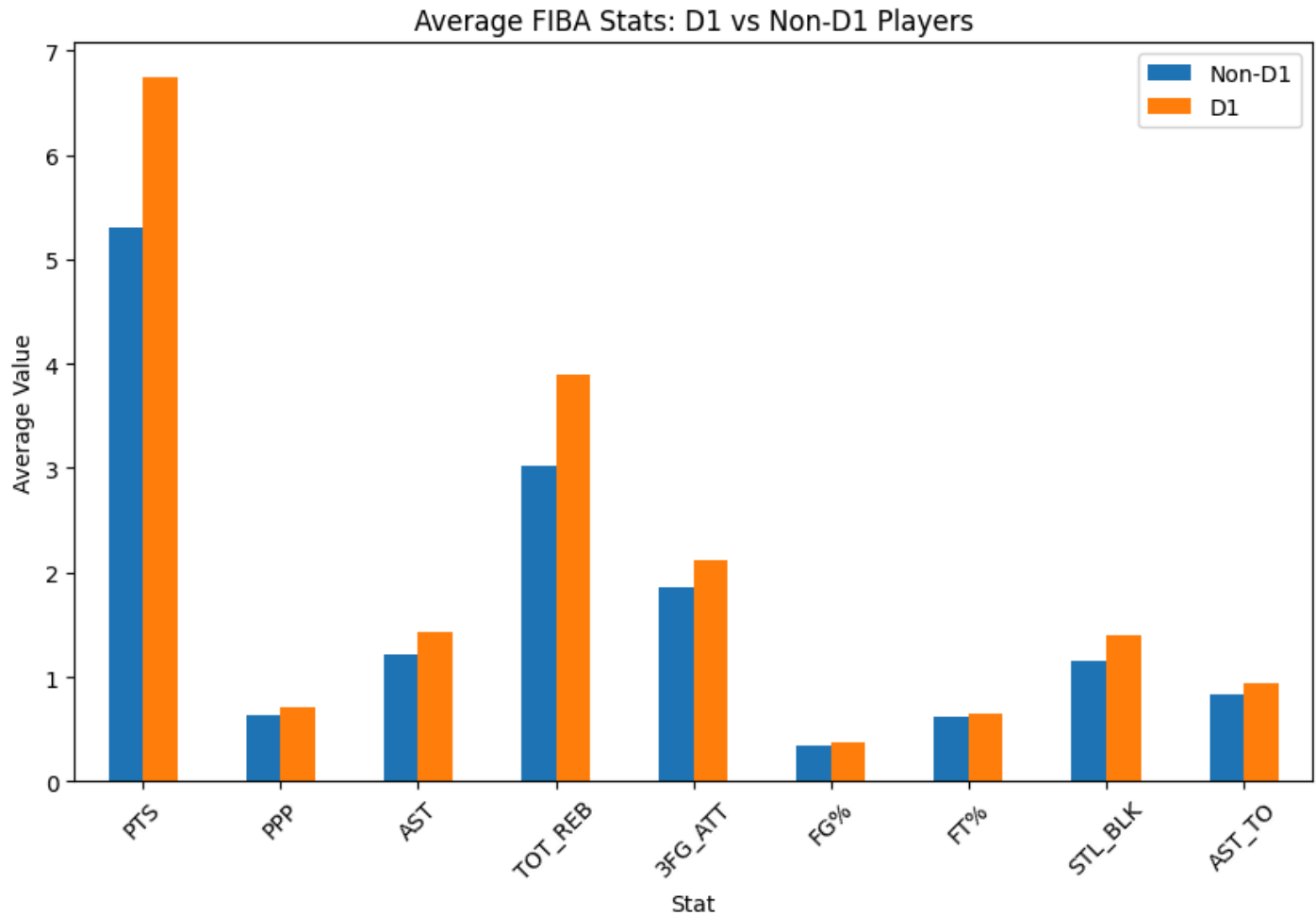
plt.title("Average FIBA Stats: D1 vs Non-D1 Players")
plt.ylabel("Average Value")
plt.xlabel("Stat")

plt.xticks(rotation=45)

plt.legend(["Non-D1", "D1"])

plt.show()

```



Stage 2 - Classification Model (FIBA->NCAA Translation)

Model A - COUNTRY INCLUDED

In [142...

```

# =====
# STAGE 2 – CLASSIFICATION MODEL (FIBA → NCAA TRANSLATION)
# Predicts probability of making NCAA Division I (made_d1)
# Includes cluster label as a feature
#
# THIS BLOCK ANSWERS GUIDING QUESTIONS:
# Q1: Which FIBA traits predict making D1?
# Q2: Do certain archetypes (clusters) convert at higher rates?
# Q3: How important are stats vs context (country, class, role)?

```

```

# =====
# -----
# FEATURE SET
# -----
model_features = cluster_features + ["ROLE", "COUNTRY", "fiba_cluster"]

X_model = df[model_features].copy()

# ROLE: categorical → treat missing as its own category
X_model["ROLE"] = X_model["ROLE"].fillna("Unknown")

y = df["made_d1"]
y.fillna(0, inplace=True)
# -----
# OPTIONAL: ADD CONTEXT FEATURES
# (comment out if you want pure stat model first)
# -----
X_model["ROLE"] = X_model["ROLE"].fillna("Unknown")
X_model = pd.get_dummies(
    X_model,
    columns = ["COUNTRY", "ROLE"],
    drop_first = True
)
X_model = pd.concat([
    X_model,
    pd.get_dummies(df['FIBA_CLASS'], prefix="class", drop_first=True)
], axis = 1)

# =====
# HANDLE MISSING VALUES
# =====

# -----
# SHOOTING PERCENTAGES
# Missing usually means no attempts
# -----
shooting_cols = ["3FG%", "FT%", "FG%", "EFG%"]

for col in shooting_cols:
    X_model[col] = X_model[col].fillna(0)

# -----
# PPP
# small amount missing
# -----
X_model["PPP"] = X_model["EFG%"] * 2
X_model["PPP"] = X_model["PPP"].fillna(X_model["PPP"].median())

# -----
# AST_TO
# undefined often because turnovers = 0
# use median or capped value
# -----
X_model["AST_TO"] = X_model["AST_TO"].fillna(X_model["AST_TO"].median())

# -----
# FINAL CHECK

```

```
# -----  
print(X_model.isna().sum().sum())
```

0

In [143...

```
# -----  
# TRAIN / TEST SPLIT  
# -----  
from sklearn.model_selection import train_test_split  
  
X_train, X_test, y_train, y_test = train_test_split(  
    X_model,  
    y,  
    test_size=0.25,  
    random_state=42,  
    stratify=y  
)  
  
# -----  
# MODEL: GRADIENT BOOSTING (STRONG BASELINE)  
# -----  
from sklearn.ensemble import GradientBoostingClassifier  
  
model = GradientBoostingClassifier(  
    n_estimators=200,  
    learning_rate=0.05,  
    max_depth=3,  
    random_state=42  
)  
  
model.fit(X_train, y_train)  
  
# -----  
# EVALUATION  
# -----  
from sklearn.metrics import classification_report, roc_auc_score  
  
y_prob = model.predict_proba(X_test)[:, 1]  
y_pred = (y_prob > 0.25).astype(int)  
  
print("Classification Report:")  
print(classification_report(y_test, y_pred))  
  
print("\nROC-AUC:")  
print(roc_auc_score(y_test, y_prob))  
  
# -----  
# FEATURE IMPORTANCE  
# -----  
importances = pd.Series(  
    model.feature_importances_,  
    index=X_model.columns  
)  
.sort_values(ascending=False)  
  
print("\nTop Features:")  
display(importances.head(19))  
  
# -----  
# CLUSTER CONVERSION RATE (KEY INSIGHT)  
# -----  
cluster_conversion = (
```

```

df.groupby("fiba_cluster")["made_d1"]
    .mean()
    .sort_index()
)

print("\nD1 Conversion Rate by Cluster:")
display((cluster_conversion * 100).round(2))
#0.25 threshold

```

Classification Report:

	precision	recall	f1-score	support
0.0	0.85	0.63	0.72	413
1.0	0.38	0.66	0.48	140
accuracy			0.64	553
macro avg	0.61	0.65	0.60	553
weighted avg	0.73	0.64	0.66	553

ROC-AUC:

0.7240747146316153

Top Features:

COUNTRY_Canada	0.156134
TOT_REB	0.130292
PTS	0.075538
POSS	0.049001
3FG_ATT	0.048624
T0	0.048139
FT%	0.046038
AST_T0	0.042041
FG%	0.039504
FT_ATT	0.038958
COUNTRY_Czechia	0.035030
STL_BLK	0.033663
3FG%	0.031419
COUNTRY_Bulgaria	0.031045
AST	0.023455
class_U17	0.022366
class_U19	0.022038
EFG%	0.019479
GP	0.018803

dtype: float64

D1 Conversion Rate by Cluster:

fiba_cluster

0 32.77

1 16.26

2 25.81

Name: made_d1, dtype: float64

Model Evaluation

In [144...

```

from sklearn.metrics import roc_curve, auc

fpr, tpr, thresholds = roc_curve(y_test, y_prob)
roc_auc = auc(fpr, tpr)

plt.figure(figsize=(6,6))

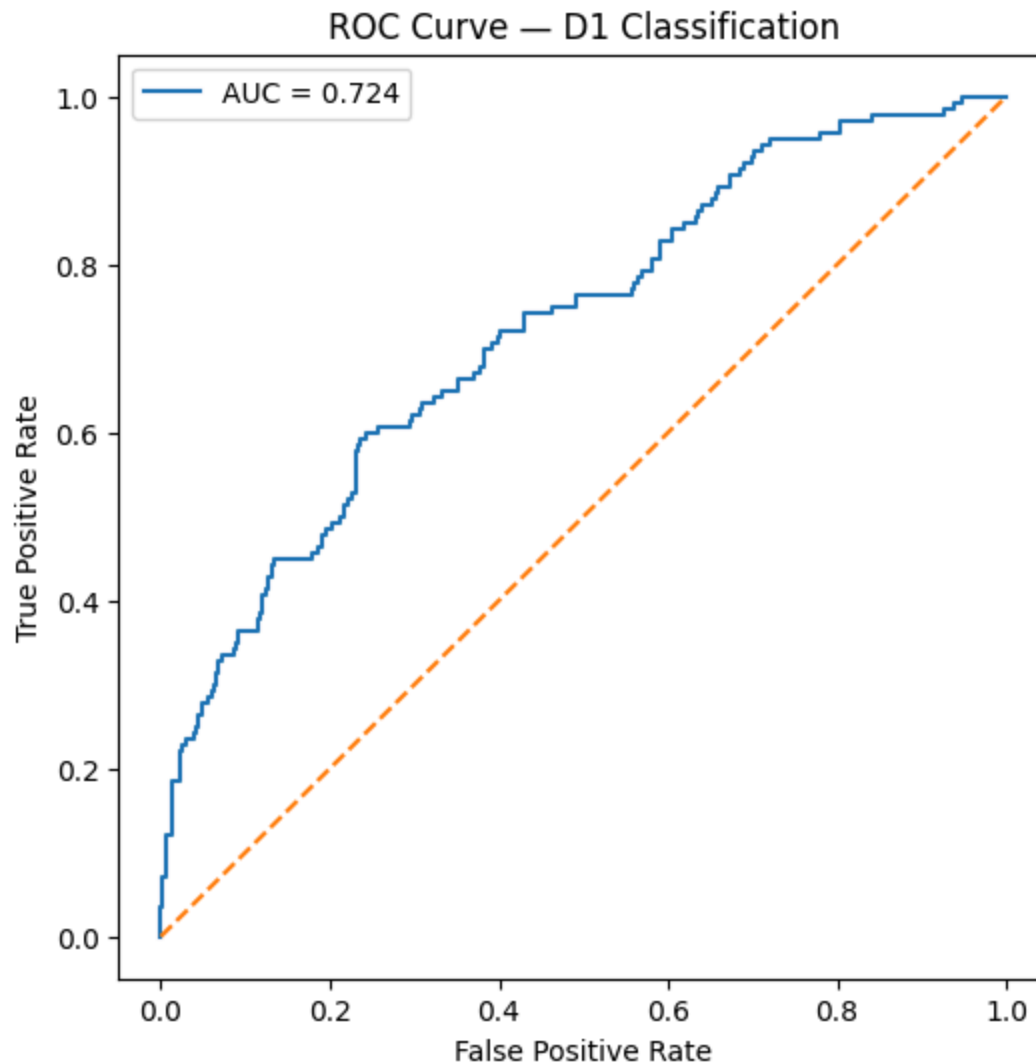
plt.plot(fpr, tpr, label=f"AUC = {roc_auc:.3f}")

```

```
plt.plot([0,1], [0,1], linestyle="--")

plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title("ROC Curve — D1 Classification")
plt.legend()

plt.show()
```



```
In [145... from sklearn.metrics import confusion_matrix
import seaborn as sns

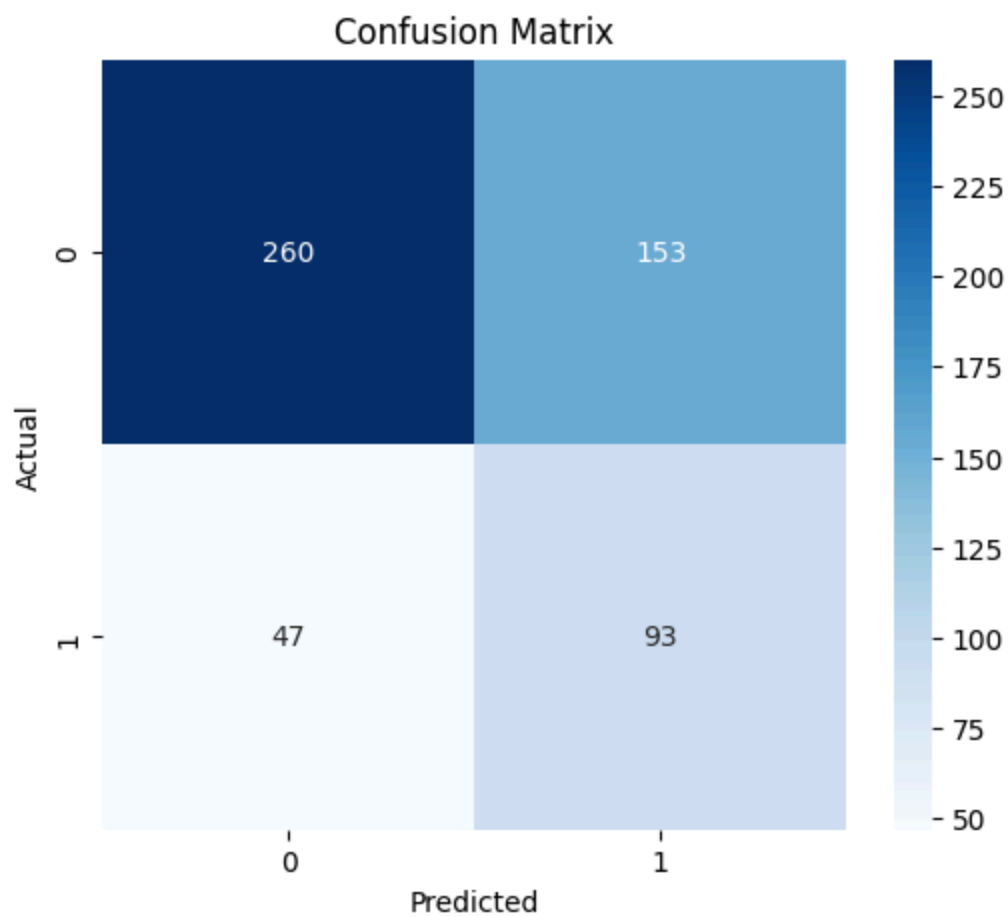
cm = confusion_matrix(y_test, y_pred)

plt.figure(figsize=(6,5))

sns.heatmap(
    cm,
    annot=True,
    fmt="d",
    cmap="Blues"
)

plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Confusion Matrix")

plt.show()
```



Feature Importance

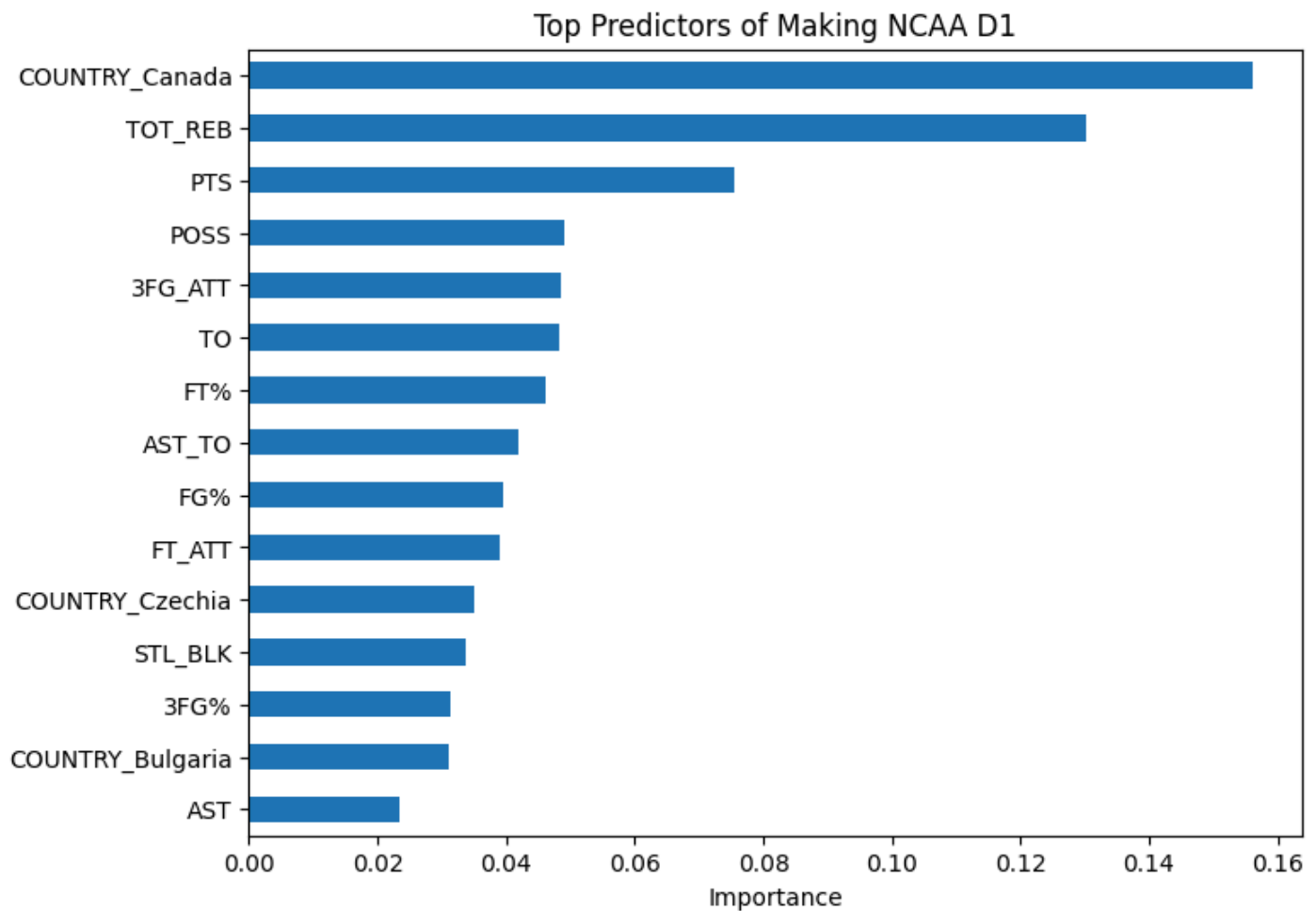
```
In [146... top_features = importances.head(15)

plt.figure(figsize=(8,6))

top_features.sort_values().plot(kind="barh")

plt.xlabel("Importance")
plt.title("Top Predictors of Making NCAA D1")

plt.show()
```

Classifier Model B - NO COUNTRY

```
In [147... # Stage 2 - Classification Model (FIBA->NCAA Translation)
# =====
# STAGE 2 - CLASSIFICATION MODEL (FIBA → NCAA TRANSLATION)
# Predicts probability of making NCAA Division I (made_d1)
# Includes cluster label as a feature
#
# THIS BLOCK ANSWERS GUIDING QUESTIONS:
# Q1: Which FIBA traits predict making D1?
# Q2: Do certain archetypes (clusters) convert at higher rates?
# Q3: How important are stats vs context (country, class, role)?
# =====

# -----
# FEATURE SET
# -----
model_featuresb = cluster_features + ["ROLE","fiba_cluster"]

X_modelb = df[model_featuresb].copy()

# ROLE: categorical → treat missing as its own category
#X_model["ROLE"] = X_model["ROLE"].fillna("Unknown")

y = df["made_d1"]
y.fillna(0, inplace=True)
# -----
# OPTIONAL: ADD CONTEXT FEATURES
# (comment out if you want pure stat model first)
# -----
```

```

X_modelb["ROLE"] = X_modelb["ROLE"].fillna("Unknown")
X_modelb = pd.get_dummies(

    X_modelb,
    columns = ["ROLE"],
    drop_first = True
)
X_modelb = pd.concat([
    X_modelb,
    pd.get_dummies(df['FIBA_CLASS'], prefix="class", drop_first=True)
], axis = 1)

# =====
# HANDLE MISSING VALUES
# =====

# -----
# SHOOTING PERCENTAGES
# Missing usually means no attempts
# -----
shooting_cols = ["3FG%", "FT%", "FG%", "EFG%"]

for col in shooting_cols:
    X_modelb[col] = X_modelb[col].fillna(0)

# -----
# PPP
# small amount missing
# -----
X_modelb["PPP"] = X_modelb["EFG%"] * 2
X_modelb["PPP"] = X_modelb["PPP"].fillna(X_modelb["PPP"].median())

# -----
# AST_TO
# undefined often because turnovers = 0
# use median or capped value
# -----

X_modelb["AST_TO"] = X_modelb["AST_TO"].fillna(X_modelb["AST_TO"].median())

# -----
# TRAIN / TEST SPLIT
# -----
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X_modelb,
    y,
    test_size=0.25,
    random_state=42,
    stratify=y
)

# -----
# MODEL: GRADIENT BOOSTING (STRONG BASELINE)
# -----
from sklearn.ensemble import GradientBoostingClassifier

modelb = GradientBoostingClassifier(
    n_estimators=200,

```

```

        learning_rate=0.05,
        max_depth=3,
        random_state=42
    )

modelb.fit(X_train, y_train)

# -----
# EVALUATION
# -----
from sklearn.metrics import classification_report, roc_auc_score

y_prob = modelb.predict_proba(X_test)[:, 1]
y_pred = (y_prob > 0.25).astype(int)

print("Classification Report:")
print(classification_report(y_test, y_pred))

print("\nROC-AUC:")
print(roc_auc_score(y_test, y_prob))

```

Classification Report:

	precision	recall	f1-score	support
0.0	0.82	0.60	0.69	413
1.0	0.34	0.60	0.43	140
accuracy			0.60	553
macro avg	0.58	0.60	0.56	553
weighted avg	0.69	0.60	0.63	553

ROC-AUC:
0.6554652369422345

In [148...

```

# -----
# FEATURE IMPORTANCE
# -----
importancesb = pd.Series(
    modelb.feature_importances_,
    index=X_modelb.columns
).sort_values(ascending=False)

print("\nTop Features:")
display(importancesb.head(11))

# -----
# CLUSTER CONVERSION RATE (KEY INSIGHT)
# -----
cluster_conversion = (
    df.groupby("fiba_cluster")["made_d1"]
    .mean()
    .sort_index()
)

print("\nD1 Conversion Rate by Cluster:")
display((cluster_conversion * 100).round(2))
#0.25 threshold

```

Top Features:

```

COUNTRY_Canada      0.156134
TOT_REB              0.130292
PTS                  0.075538
POSS                 0.049001
3FG_ATT              0.048624
TO                   0.048139
FT%                  0.046038
AST_TO               0.042041
FG%                  0.039504
FT_ATT              0.038958
COUNTRY_Czechia     0.035030
dtype: float64
D1 Conversion Rate by Cluster:
fiba_cluster
0      32.77
1      16.26
2      25.81
Name: made_d1, dtype: float64

```

```

In [149... ### Model Evaluation
from sklearn.metrics import roc_curve, auc

fpr, tpr, thresholds = roc_curve(y_test, y_prob)
roc_auc = auc(fpr, tpr)

plt.figure(figsize=(4,4))

plt.plot(fpr, tpr, label=f"AUC = {roc_auc:.3f}")
plt.plot([0,1], [0,1], linestyle="--")

plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title("ROC Curve - D1 Classification")
plt.legend()

plt.show()
from sklearn.metrics import confusion_matrix
import seaborn as sns

cm = confusion_matrix(y_test, y_pred)

plt.figure(figsize=(4,3))

sns.heatmap(
    cm,
    annot=True,
    fmt="d",
    cmap="Blues"
)

plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Confusion Matrix")

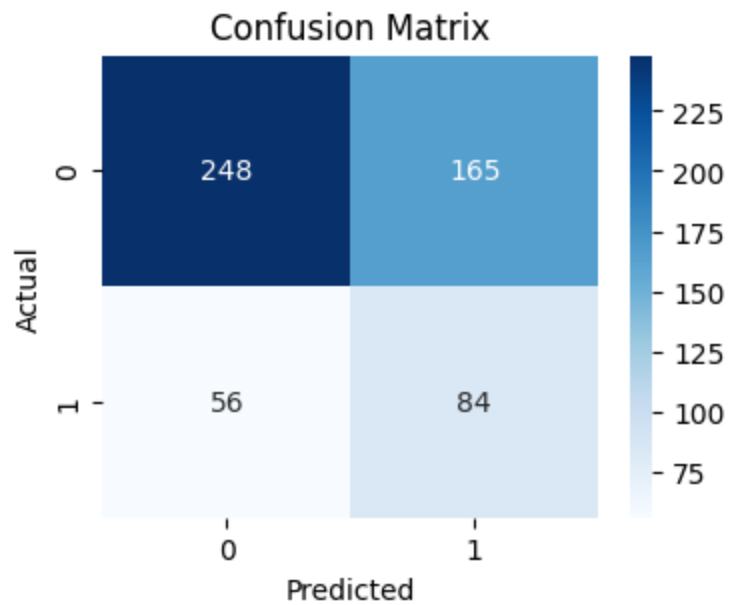
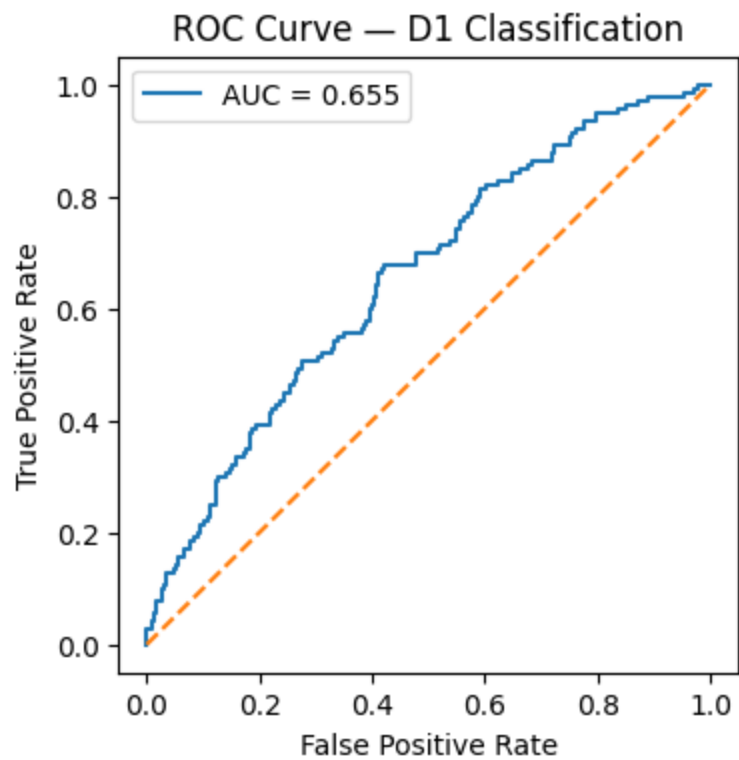
plt.show()
### Feature Importance
top_featuresb = importancesb.head(15)

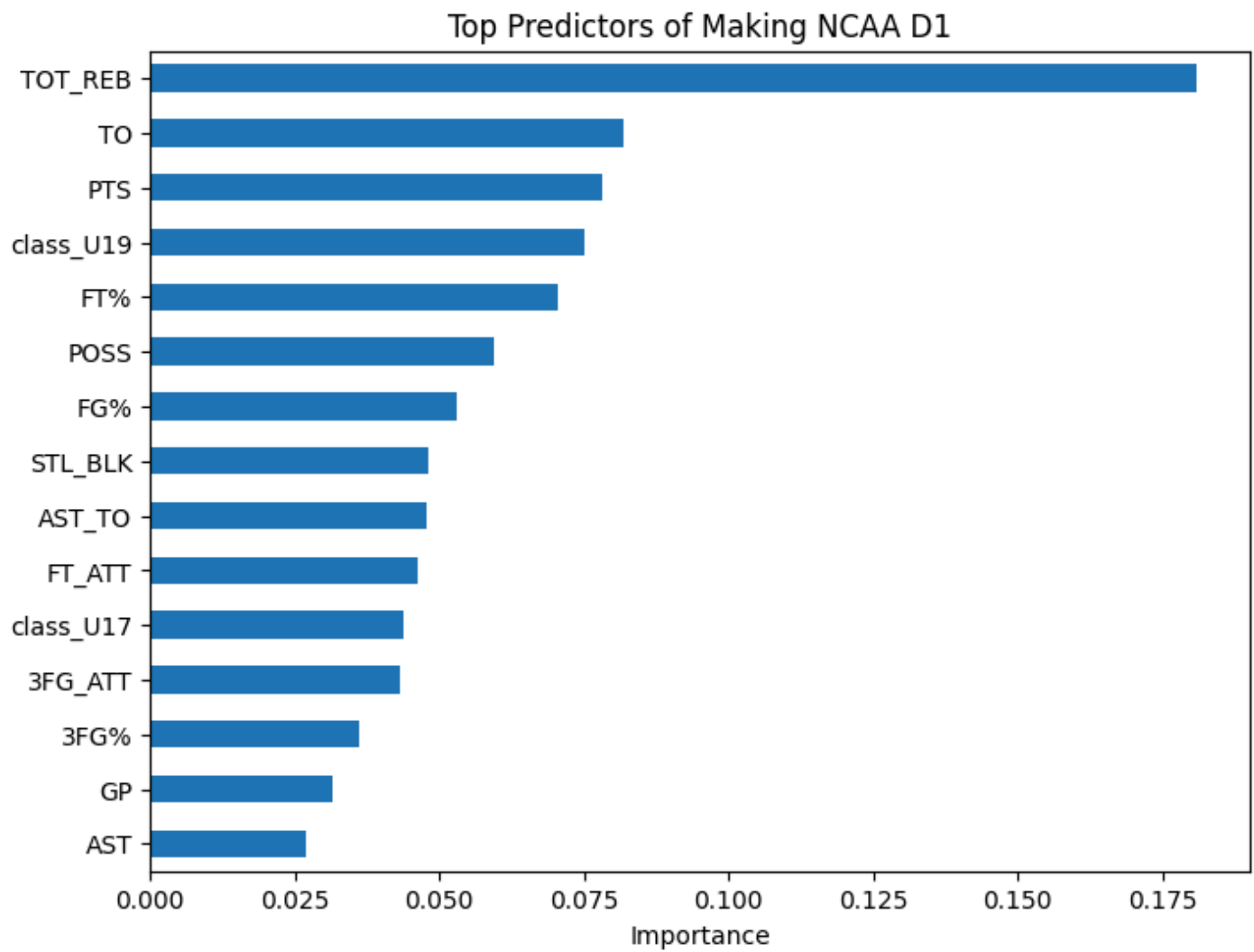
plt.figure(figsize=(8,6))

top_featuresb.sort_values().plot(kind="barh")

```

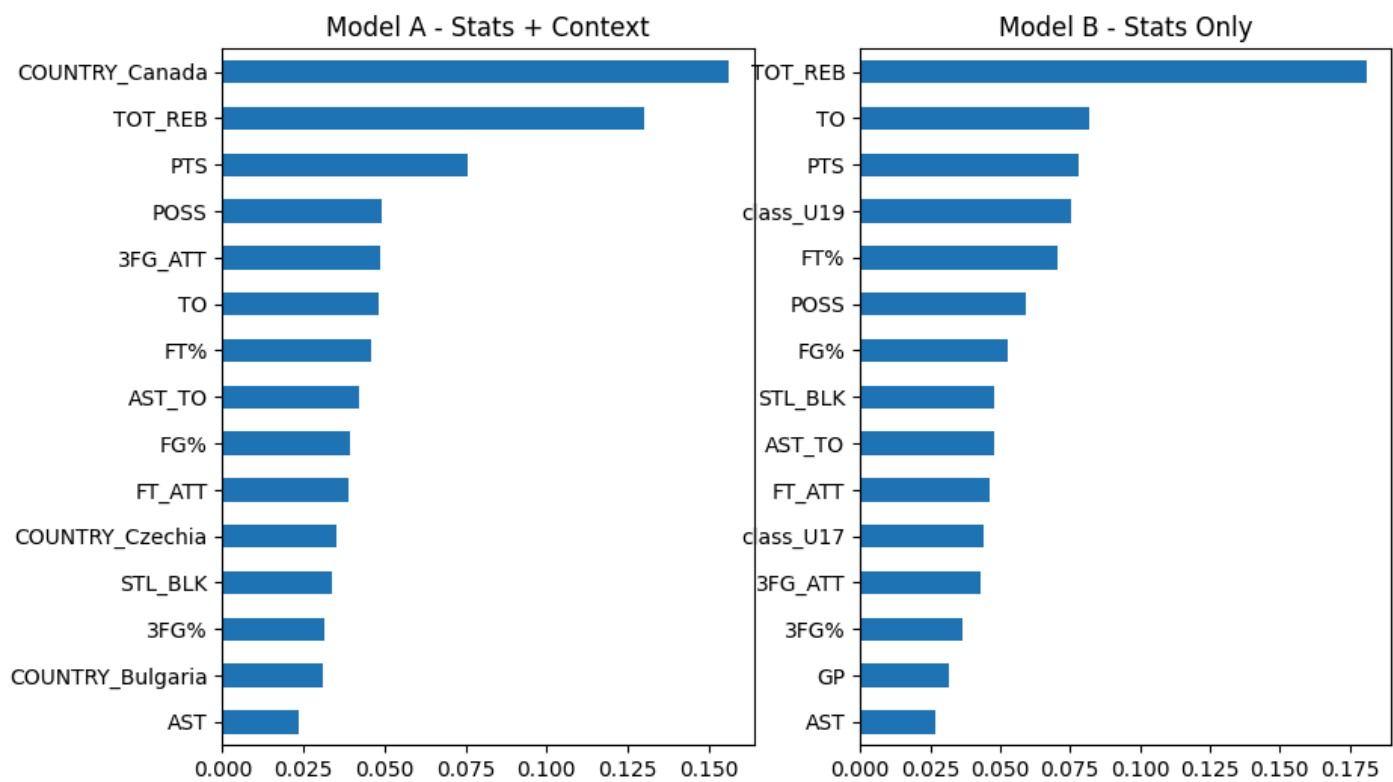
```
plt.xlabel("Importance")  
plt.title("Top Predictors of Making NCAA D1")  
  
plt.show()
```





```
In [150... # plot importances vs importancesb
plt.figure(figsize=(10,6))
ax = plt.subplot(1,2,1)
top_features.sort_values().plot(kind="barh")
plt.title("Model A - Stats + Context")
ax = plt.subplot(1,2,2)
top_featuresb.sort_values().plot(kind="barh")
plt.title("Model B - Stats Only")
```

```
Out[150... Text(0.5, 1.0, 'Model B - Stats Only')
```



Stage 3 - Regression

Describe the target variable - league_adj_best

```
In [151... print(df["league_adj_best"].describe())
```

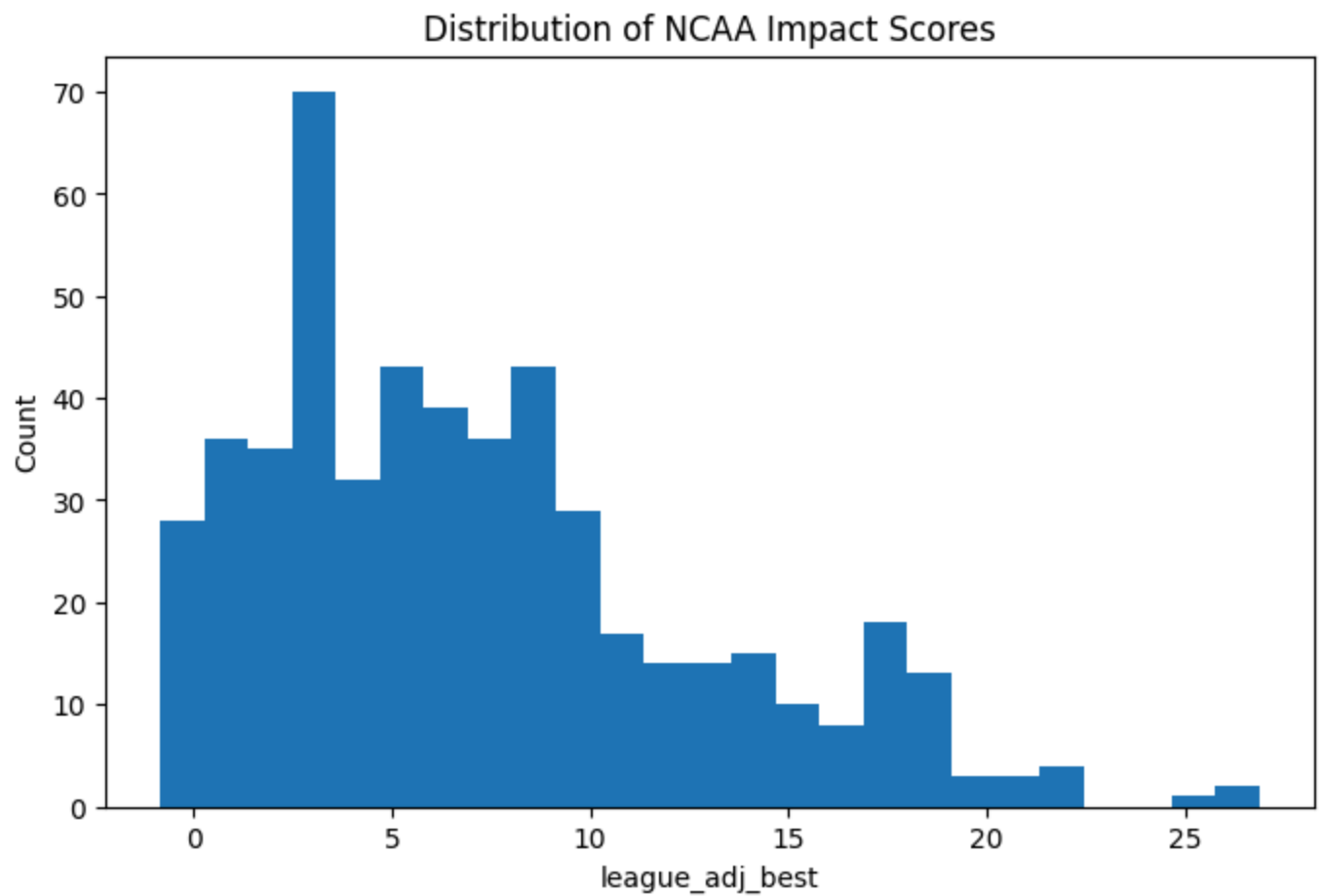
```
count    513.000000
mean      7.245148
std       5.511801
min      -0.835000
25%       2.854000
50%       6.234000
75%      10.058000
max      26.860000
Name: league_adj_best, dtype: float64
```

```
In [152... plt.figure(figsize=(8,5))

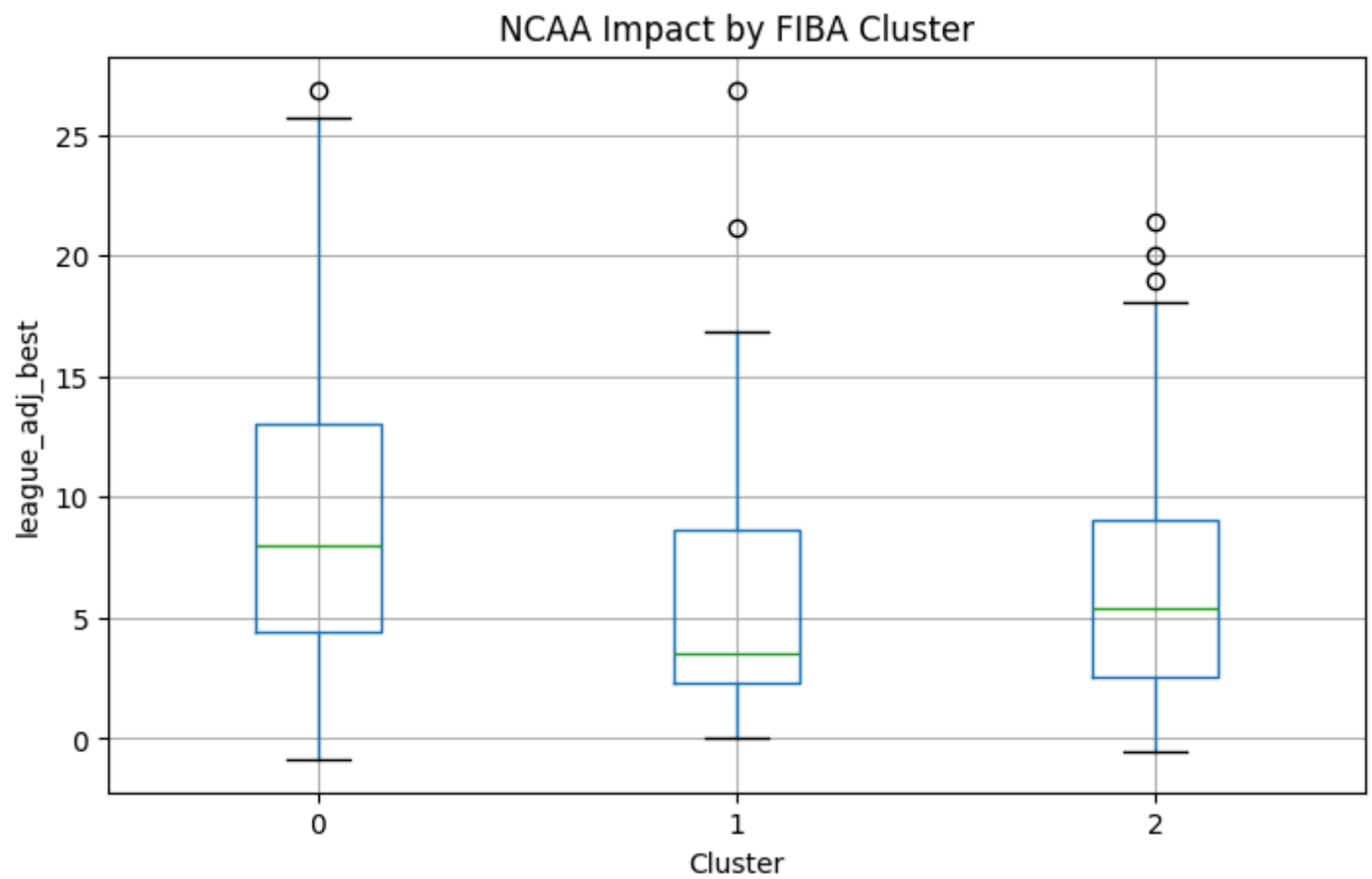
plt.hist(df["league_adj_best"], bins=25)

plt.xlabel("league_adj_best")
plt.ylabel("Count")
plt.title("Distribution of NCAA Impact Scores")

plt.show()
```



```
In [153... df.boxplot(  
    column="league_adj_best",  
    by="fiba_cluster",  
    figsize=(8,5)  
)  
  
plt.title("NCAA Impact by FIBA Cluster")  
plt.suptitle("")  
plt.xlabel("Cluster")  
plt.ylabel("league_adj_best")  
  
plt.show()
```

```
In [154... conf_summary = (  
    df.groupby("bestconf")["league_adj_best"]  
    .agg(["count", "mean"])  
    .sort_values("mean", ascending=False)  
)  
  
display(conf_summary.head(15))
```

	count	mean
bestconf		
ACC	33	10.291333
Big Ten	44	9.882250
Horizon	5	9.553800
WCC	67	9.091433
SEC	32	8.760188
Pac-12	17	8.551353
SWAC	1	8.532000
Southern	4	8.356750
Big East	29	7.872586
A-10	34	7.001529
MAAC	13	6.744077
CUSA	14	6.694929
Patriot	9	6.650111
CAA	7	6.469429
Sun Belt	4	6.465000

```
In [155... df_clusters = df[df['made_d1']==1]
df_clusters[['NAME', 'FIBA_YEAR', 'fiba_cluster']].to_csv("fiba_clusters.csv", index = False)
```

```
In [156... df_reg2 = df[
    (df["made_d1"] == 1) &
    (~df["league_adj_best"].isna())
].copy()

df_reg2["impact_bucket"] = pd.cut(
    df_reg2["league_adj_best"],
    bins=[0,5,15,100],
    labels=["Low (<5)", "Mid (5-15)", "High (15+)"]
)
bucket_summary = (
    df_reg2.groupby("impact_bucket")[compare_stats]
    .mean()
)

plt.figure(figsize=(10,6))

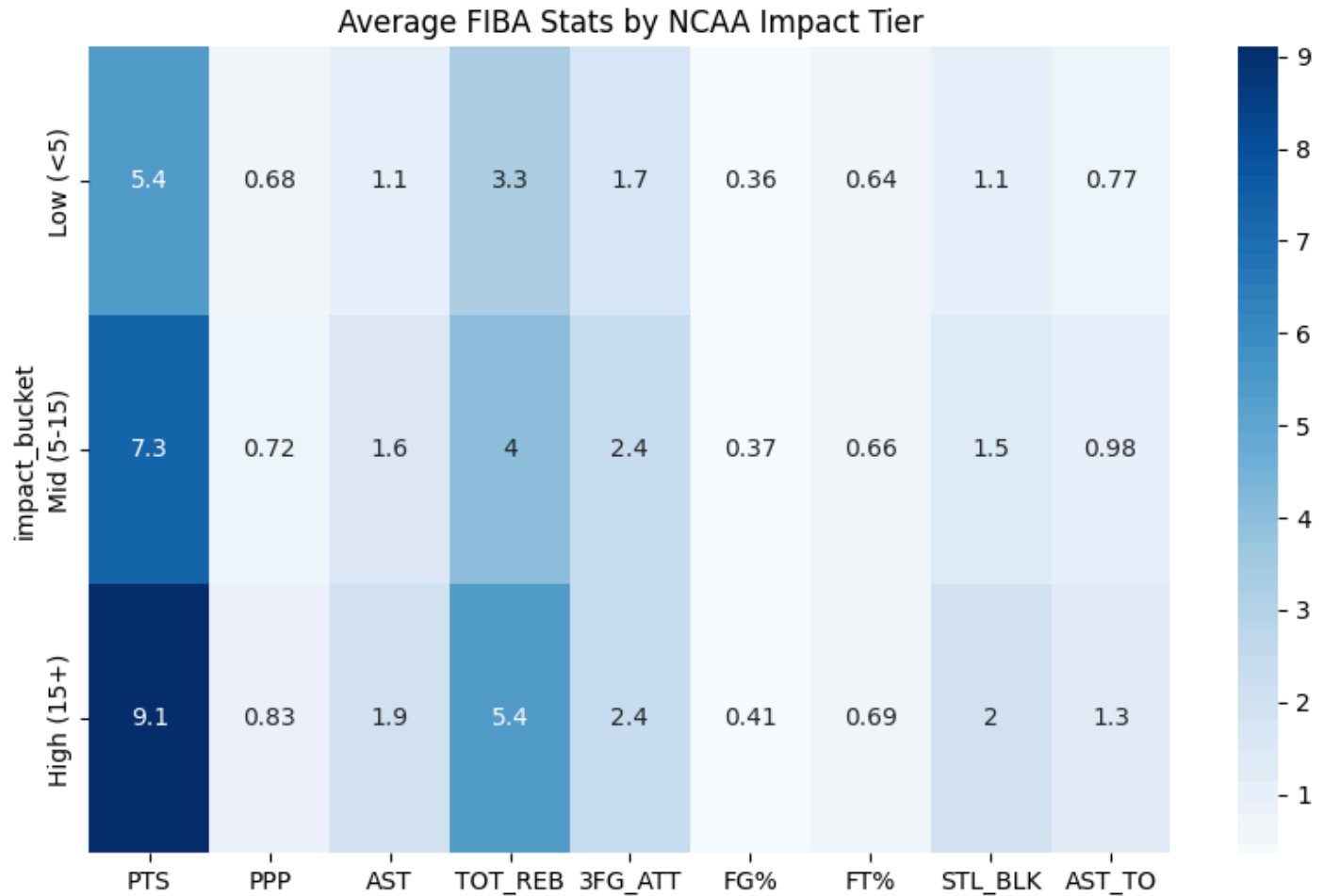
sns.heatmap(
    bucket_summary,
    annot=True,
    cmap="Blues"
)

plt.title("Average FIBA Stats by NCAA Impact Tier")

plt.show()
```

/var/folders/qh/zhr4gq8s5px2ff46zwfdk90hy3y30j/T/ipykernel_29369/903129192.py:12: FutureWarning: The default of observed=False is deprecated and will be changed to True in a future version of pandas. Pass observed=False to retain current behavior or observed=True to adopt the future default and silence this warning.

```
df_reg2.groupby("impact_bucket")[compare_stats]
```



```
In [157... # print [['NAME', 'FIBA_YEAR', 'league_adj_best', 'impact_bucket']].sample(10)
print(
    df_reg2.loc[
        df_reg2["impact_bucket"] == "High (15+)",
        ["NAME", "FIBA_YEAR", "COUNTRY", "league_adj_best", "impact_bucket", "bestconf"]
    ].sample(11)
)
```

	NAME	FIBA_YEAR	COUNTRY	league_adj_best	impact_bucket	bestconf
38	Skylar Forbes	2022	Canada	17.287	High (15+)	Big East
8	Cearah Parchment	2024	Canada	21.131	High (15+)	Big Ten
59	Olivia Pollerd	2018	Australia	15.136	High (15+)	WCC
15	Fatmata Jannet	2019	Great Britain	18.906	High (15+)	SEC
13	Mama Dembele	2019	Spain	18.943	High (15+)	SEC
50	Freja Werth	2019	Sweden	15.925	High (15+)	WCC
20	Cassandre Prosper	2023	Canada	18.845	High (15+)	ACC
10	Georgia Amoore	2018	Australia	20.019	High (15+)	SEC
49	Mila Holloway	2025	Canada	16.055	High (15+)	Big Ten
17	Fatmata Jannet	2023	Great Britain	18.906	High (15+)	SEC
43	Elena Rodriguez	2023	Spain	16.935	High (15+)	Ivy

Regression Model

```
In [158... # =====
# STAGE 3 – REGRESSION MODEL (FIBA → NCAA IMPACT)
# Predicts NCAA success (league_adj_best) for players who made D1
```

```

#
# THIS BLOCK ANSWERS GUIDING QUESTIONS:
# Q1: Which FIBA traits predict NCAA success?
# Q2: Which archetypes translate best after reaching D1?
# Q3: How well can we estimate player impact from FIBA stats?
# =====

df_reg = df[
    (df["made_d1"] == 1) &
    (~df["league_adj_best"].isna())
].copy()

print("Regression Sample Size:", df_reg.shape)

X_reg = X_model.copy()
X_reg = X_reg.loc[df_reg.index] # align with regression sample
y_reg = df_reg["league_adj_best"]

# -----
# TRAIN / TEST SPLIT
# -----
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X_reg,
    y_reg,
    test_size=0.25,
    random_state=42
)

# -----
# MODEL: GRADIENT BOOSTING REGRESSOR
# -----
from sklearn.ensemble import GradientBoostingRegressor

reg_model = GradientBoostingRegressor(
    n_estimators=250,
    learning_rate=0.05,
    max_depth=3,
    random_state=42
)

reg_model.fit(X_train, y_train)

# -----
# EVALUATION
# -----
from sklearn.metrics import mean_absolute_error, r2_score

y_pred = reg_model.predict(X_test)

print("MAE:", mean_absolute_error(y_test, y_pred))
print("R2:", r2_score(y_test, y_pred))

# -----
# FEATURE IMPORTANCE
# -----
importances = pd.Series(

```

```

        reg_model.feature_importances_,
        index=X_reg.columns
    ).sort_values(ascending=False)

print("\nTop Features:")
display(importances.head(15))

# -----
# CLUSTER PERFORMANCE INSIGHT
# -----
cluster_success = (
    df_reg.groupby("fiba_cluster")["league_adj_best"]
    .mean()
    .sort_index()
)

print("\nAvg NCAA Impact by Cluster:")
display(cluster_success.round(2))

```

Regression Sample Size: (513, 40)
MAE: 3.876797882916573
R2: 0.2763653989386359

Top Features:

PTS	0.163111
TOT_REB	0.084965
COUNTRY_Canada	0.081003
AST_TO	0.066953
AST	0.061904
STL_BLK	0.055347
POSS	0.042415
class_U19	0.041909
FG%	0.040558
TO	0.038921
FT_ATT	0.037029
COUNTRY_Spain	0.035948
FT%	0.035934
GP	0.032366
3FG_ATT	0.026899

dtype: float64

Avg NCAA Impact by Cluster:

fiba_cluster

0 8.98

1 5.65

2 6.35

Name: league_adj_best, dtype: float64

Regression Model B - No Country

In [159...

```

# =====
# STAGE 3 – REGRESSION MODEL (FIBA → NCAA IMPACT)
# Predicts NCAA success (league_adj_best) for players who made D1
#
# THIS BLOCK ANSWERS GUIDING QUESTIONS:
# Q1: Which FIBA traits predict NCAA success?
# Q2: Which archetypes translate best after reaching D1?
# Q3: How well can we estimate player impact from FIBA stats?
# =====

```

```
df_reg = df[
```

```

(df["made_d1"] == 1) &
(~df["league_adj_best"].isna())
].copy()

print("Regression Sample Size:", df_reg.shape)

X_regb = X_modelb.copy()
X_regb = X_regb.loc[df_reg.index] # align with regression sample
y_reg = df_reg["league_adj_best"]

# -----
# TRAIN / TEST SPLIT
# -----
from sklearn.model_selection import train_test_split

X_trainb, X_testb, y_trainb, y_testb = train_test_split(
    X_regb,
    y_reg,
    test_size=0.25,
    random_state=42
)

# -----
# MODEL: GRADIENT BOOSTING REGRESSOR
# -----
from sklearn.ensemble import GradientBoostingRegressor

reg_modelb = GradientBoostingRegressor(
    n_estimators=250,
    learning_rate=0.05,
    max_depth=3,
    random_state=42
)

reg_modelb.fit(X_trainb, y_trainb)

# -----
# EVALUATION
# -----
from sklearn.metrics import mean_absolute_error, r2_score

y_predb = reg_modelb.predict(X_testb)

print("MAE:", mean_absolute_error(y_testb, y_predb))
print("R2:", r2_score(y_testb, y_predb))

# -----
# FEATURE IMPORTANCE
# -----
importancesb = pd.Series(
    reg_modelb.feature_importances_,
    index=X_regb.columns
).sort_values(ascending=False)

print("\nTop Features:")
display(importancesb.head(15))

```

Regression Sample Size: (513, 40)
MAE: 4.124932751183817
R2: 0.18131914489227974

Top Features:

PTS	0.171632
AST_T0	0.102862
class_U19	0.082890
AST	0.081282
STL_BLK	0.070169
TOT_REB	0.067449
POSS	0.065223
FT_ATT	0.053477
FG%	0.042412
GP	0.035199
3FG_ATT	0.033675
ROLE_Spot-up shooting wing	0.031126
3FG%	0.028635
FT%	0.026496
T0	0.023882

dtype: float64

In []:

Project Overview / Final Takeaways

In [164]...

```
# =====  
# PROJECT OVERVIEW / FINAL TAKEAWAYS SECTION  
# =====  
  
print("\n" + "="*70)  
print("PROJECT OVERVIEW")  
print("="*70)  
  
print("""  
Goal:  
Develop a data pipeline that uses international FIBA youth data  
to identify which players are most likely to become impactful  
NCAA Division I women's basketball players.  
  
Core Question:  
Can we use international performance metrics to predict future  
college basketball success?  
""")  
  
print("\nKey Components Built:")  
components = [  
    "FIBA youth tournament scraper + cleaner",  
    "Player identity matching pipeline",  
    "League-adjusted competition strength metric",  
    "Career NCAA statistics integration",  
    "Feature engineering + role metrics",  
    "Regression + classification modeling",  
    "Player impact bucket segmentation",  
    "Visualization + exploratory analysis"  
]  
  
for c in components:  
    print(f"• {c}")
```

```

print("\n" + "="*70)
print("DATASET SUMMARY")
print("="*70)

print(f"Total Players: {len(df):,}")
print(f"Players Reaching D1: {df['made_d1'].sum():,}")

if 'impact_bucket' in df_reg2.columns:
    print("\nImpact Bucket Distribution:")
    print(df_reg2['impact_bucket'].value_counts())

```

PROJECT OVERVIEW

Goal:

Develop a data pipeline that uses international FIBA youth data to identify which players are most likely to become impactful NCAA Division I women's basketball players.

Core Question:

Can we use international performance metrics to predict future college basketball success?

Key Components Built:

- FIBA youth tournament scraper + cleaner
- Player identity matching pipeline
- League-adjusted competition strength metric
- Career NCAA statistics integration
- Feature engineering + role metrics
- Regression + classification modeling
- Player impact bucket segmentation
- Visualization + exploratory analysis

DATASET SUMMARY

Total Players: 2,211
Players Reaching D1: 561.0

Impact Bucket Distribution:

```

impact_bucket
Mid (5-15)    241
Low (<5)      194
High (15+)    60
Name: count, dtype: int64

```

In [167...

```

print("\n" + "="*70)
print("KEY MODEL TAKEAWAYS")
print("="*70)

takeaways = [

    "The D1 vs Non-D1 comparison chart showed that the gaps in performance were marginal",

    "This dataset doesn't account for FIBA players who never made it to the NCAA, which",

    "This dataset doesn't account for FIBA players who went to other international leagues",

    "This dataset doesn't account for players who played junior college (JUCO) or Division I",

```



```

    "This dataset looks at FIBA tournament stats which is generally 6 to 8 games per pla
    "Raw production metrics consistently outperformed efficiency metrics as predictors o
    "Rebounding and scoring volume were the strongest recurring indicators across both r
    "Model A (Stats + Context) consistently outperformed Model B (Stats Only), showing t
    "Country added predictive value beyond traditional box score statistics alone.",
    "The feature importance charts showed TOT_REB and PTS as dominant predictors across
    "Efficiency statistics such as PPP, 3PT%, and FT% had significantly lower predictive
    "The NCAA impact tier heatmap showed a strong progression pattern: higher-impact pla
    "The distribution of league_adj_best was highly right-skewed, with most players clus
    "This high-variance outcome structure likely contributed to the moderate regression
    "The classification pipeline produced stronger and more stable results than the regr
    "Recall-optimized classification models were particularly valuable because they prio
    "From a scouting perspective, missing a future high-level player is often more costl
    "The regression models captured broad relationships between international youth prod
    "Overall, the models suggest that early international production profiles contain me
]

```

```

for i, t in enumerate(takeaways, 1):
    print(f"{i}. {t}")

```

```

print("\n" + "="*70)
print("PROJECT IMPACT")
print("="*70)

```

```

print("""
This project demonstrates that international youth basketball data is inherently noisy a

```

The strongest finding was not simply that statistics matter –
but that context matters alongside production.

Adding contextual variables substantially improved model performance,
showing that evaluating players within their competition environment
provides a more complete understanding of future development.

The project also demonstrated the value of recall-focused classification
for scouting applications. In talent identification settings, successfully
capturing future D1 players is often more important than minimizing
false positives.

The pipeline can support:

- International scouting
- Prospect identification
- Recruiting prioritization

Future improvements could include:

- Multi-year progression modeling
 - Expanded non-NCAA international datasets
- ```
""")
```

```
print("\n" + "="*70)
print("FINAL CONCLUSION")
print("="*70)
```

```
print("""
International youth basketball statistics alone do not fully explain
future NCAA success – but when combined with contextual information,
they become substantially more predictive.
```

Across both regression and classification modeling, the most reliable signals were consistent production indicators such as scoring and rebounding, while traditional efficiency metrics played a much smaller role than expected.

The project also demonstrated the value of recall-focused classification for scouting applications. In talent identification settings, successfully capturing future D1 players is often more important than minimizing false positives. Moreover, most players are clustered in the low-to-mid impact range, making it inherently difficult to predict exact outcomes.

```
""")
```

## KEY MODEL TAKEAWAYS

1. The D1 vs Non-D1 comparison chart showed that the gaps in performance were marginal. Therefore making accurate predictions is inherently difficult, especially with noisy developmental pathways and limited sample sizes.
2. This dataset doesn't account for FIBA players who never made it to the NCAA, which likely introduces survivorship bias and limits the ability to fully capture the translation process.
3. This dataset doesn't account for FIBA players who went to other international leagues or professional paths, which may limit the generalizability of the findings to all youth international players.
4. This dataset doesn't account for players who played junior college (JUCO) or Division II, which may limit the ability to capture all developmental pathways to D1.
5. This dataset looks at FIBA tournament stats which is generally 6 to 8 games per player, which is a small sample size and may not fully capture a player's true ability or potential.
6. Raw production metrics consistently outperformed efficiency metrics as predictors of future NCAA success.
7. Rebounding and scoring volume were the strongest recurring indicators across both regression and classification models.
8. Model A (Stats + Context) consistently outperformed Model B (Stats Only), showing that contextual variables meaningfully improve player evaluation.
9. Country added predictive value beyond traditional box score statistics alone.
10. The feature importance charts showed TOT\_REB and PTS as dominant predictors across nearly every modeling approach.
11. Efficiency statistics such as PPP, 3PT%, and FT% had significantly lower predictive importance than expected.
12. The NCAA impact tier heatmap showed a strong progression pattern: higher-impact players consistently produced more points, rebounds, assists, and steals/blocks at the FIBA level.
13. The distribution of league\_adj\_best was highly right-skewed, with most players clustering in the low-to-mid impact range and a smaller subset reaching elite outcomes.
14. This high-variance outcome structure likely contributed to the moderate regression performance, as predicting exact long-term NCAA impact from youth international data is inherently noisy.
15. The classification pipeline produced stronger and more stable results than the regression models, suggesting that identifying future NCAA players is more tractable than predicting exact collegiate impact levels.
16. Recall-optimized classification models were particularly valuable because they prioritized identifying future D1 players, even at the cost of additional false positives.
17. From a scouting perspective, missing a future high-level player is often more costly than over-identifying a prospect, making recall optimization strategically important.
18. The regression models captured broad relationships between international youth production and eventual NCAA impact level, even with noisy developmental pathways.
19. Overall, the models suggest that early international production profiles contain meaningful long-term signal when paired with contextual information.

## PROJECT IMPACT

This project demonstrates that international youth basketball data is inherently noisy and incomplete, but still contains meaningful signals about future NCAA success when combined with contextual information.

The strongest finding was not simply that statistics matter – but that context matters alongside production.

Adding contextual variables substantially improved model performance, showing that evaluating players within their competition environment provides a more complete understanding of future development.

The project also demonstrated the value of recall-focused classification for scouting applications. In talent identification settings, successfully capturing future D1 players is often more important than minimizing false positives.

The pipeline can support:

- International scouting
- Prospect identification
- Recruiting prioritization

Future improvements could include:

- Multi-year progression modeling
- Expanded non-NCAA international datasets

## FINAL CONCLUSION

International youth basketball statistics alone do not fully explain future NCAA success – but when combined with contextual information, they become substantially more predictive.

Across both regression and classification modeling, the most reliable signals were consistent production indicators such as scoring and rebounding, while traditional efficiency metrics played a much smaller role than expected.

The project also demonstrated the value of recall-focused classification for scouting applications. In talent identification settings, successfully capturing future D1 players is often more important than minimizing false positives. Moreover, most players are clustered in the low-to-mid impact range, making it inherently difficult to predict exact outcomes.

In [ ]: